



Carrera:	Ingeniería en Sistemas de Información	Código:	1890	VARIANTE: A CALIFICACION: Absoluta: / 15 Relativa: / 100 Vo. Bo:	
Asignatura:	Programación II	Código:			
Ciclo:	2026	Fecha:	24 de septiembre de 2026		
Catedrático:	ING. RICHARD ORTIZ SASVIN	Jornada:			
Semestre:	2do.	Sección:	3		
Duración del examen:				 min	
EXAMEN:		SEGUNDO PARCIAL:	☒	PRIMER PARCIAL:	
EXTRAORDINARIO:		FINAL:	☐	RECUPERACIÓN:	☐

Estudiante: Mercedes María José Vásquez Méndez _____ Carné: 0907-24-6568 _____

SEGUNDA EVALUACIÓN PARCIAL DE PROGRAMACIÓN II

Semanas 7 a 11 | Python, PostgreSQL, SQLAlchemy y proyecto HelpDesk EDU

Instrucciones generales

Resuelva individualmente. Serie I: 25 preguntas de 0.20 puntos, total 5 puntos. Serie II: cinco ejercicios de 2 puntos, total 10 puntos. Puntaje máximo: 15 puntos.

En selección múltiple marque una opción. En verdadero o falso justifique únicamente las falsas. En la práctica modifique su copia del proyecto trabajado en clase, conserve su estructura y entregue código, pruebas y README_parcial2.md con archivos modificados y resultados de ejecución.

Use PostgreSQL para el ejercicio SQL. SQLite en memoria es válido para las pruebas de SQLAlchemy. Ejecute uv run pytest -q y documente cualquier fallo previo y posterior.

SERIE I Preguntas 5 puntos

A. Selección múltiple. Preguntas 1 a 15, 0.20 puntos cada una (3 puntos).

1. Role es un StrEnum. ¿Qué devuelve Role.TECHNICIAN == "technician"?

- A. False, porque son tipos distintos B. Lanza TypeError
C. True, porque StrEnum compara como texto D. None

Respuesta: C

2. HistoryEvent.created_at se define como field(default_factory=lambda: datetime.now().astimezone()) en lugar de created_at: datetime = datetime.now().astimezone(). ¿Por qué?

- A. Porque un valor por defecto evaluado una sola vez al definir la clase se compartiría entre instancias B. Por estética, no cambia el comportamiento
C. Porque datetime.now() no existe en dataclasses D. Porque field es obligatorio en todo dataclass

Respuesta: A

3. TicketService._init_(self, repository, users, notifier=None) recibe el repositorio y el servicio de usuarios como parámetros en lugar de crearlos internamente. Este patrón se llama:

- A. Herencia múltiple B. Sobrecarga de operadores
C. Encapsulamiento D. Inyección de dependencias

Respuesta: D

4. En un dataclass, ¿por qué se usa field(default_factory=list) para la colección comments?

- A. Porque ordena automáticamente sus elementos. B. Porque crea una lista por instancia; usar = [] como valor por defecto provoca ValueError.
C. Porque convierte la lista en una tupla. D. Porque todas las instancias deben usar una sola lista.

Respuesta: B

5. En assign(), self._users.require(technician_id) lanza UserNotFoundError si el id no existe. La ventaja frente a devolver None silenciosamente es:

- A. El error se detecta en el punto exacto de la falla, no más adelante con un AttributeError confuso B. Ninguna, ambos enfoques son equivalentes
C. None ocupa más memoria D. Devolver None es obligatorio en Python

Respuesta: A



UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA

FACULTAD DE INGENIERÍA EN SISTEMAS DE INFORMACIÓN

SERIE I Continuación

6. class TicketNotFoundError(NotFoundError) y class NotFoundError(DomainError). Un bloque except DomainError: en el código que llama al servicio:

- A. No captura TicketNotFoundError porque el nombre es distinto B. Lanza un error de sintaxis
C. Captura TicketNotFoundError, porque hereda de DomainError D. Solo captura excepciones nativas de Python

Respuesta: C

7. TicketRepository(ABC) declara add, by_id, list, next_id y update como @abstractmethod. ¿Qué ocurre al ejecutar TicketRepository() directamente?

- A. Se crea normalmente, vacío B. Lanza TypeError porque tiene métodos abstractos sin implementar
C. Lanza ImportError D. No ocurre nada hasta que se llame un método

Respuesta: B

8. def list(self, *, status=None, assignee_id=None, requester_id=None). El * al inicio de los parámetros:

- A. Acepta cualquier cantidad de argumentos posicionales B. Es un error de sintaxis
C. Desempaqueta una tupla D. Obliga a pasar los filtros por nombre (status=...), no posicionalmente

Respuesta: D

9. comments.ticket_id INTEGER REFERENCES tickets(id) ON DELETE CASCADE. Al ejecutar DELETE FROM tickets WHERE id = 3:

- A. Se impide el borrado si hay comentarios B. Los comentarios quedan huérfanos
C. Los comentarios del ticket 3 se borran automáticamente D. Se borra también la tabla comments

Respuesta: C

10. ¿Cuándo podría beneficiar un índice sobre tickets(status)?

- A. Cuando las consultas y la selectividad del filtro justifican su uso según el plan de ejecución. B. Siempre acelera INSERT.
C. Cuando se necesita reemplazar la clave primaria. D. Siempre reduce el tamaño de la tabla.

Respuesta: A

11. Un modelo tiene requester_id y assignee_id como FK a users.id. Al agregar una relationship hacia User, ¿qué permite desambiguar el vínculo?

- A. Eliminar las dos claves foráneas. B. Convertir users en una lista.
C. Usar únicamente commit(). D. Indicar la columna apropiada en foreign_keys.

Respuesta: D

12. SQLAlchemyTicketRepository.by_id() construye un Ticket (dataclass) nuevo con _ticket_to_domain(row) en vez de devolver la fila ORM directamente. Esto es necesario porque:

- A. Es solo una preferencia de estilo B. Sin esa traducción, TicketService tendría que conocer detalles de SQLAlchemy, rompiendo la separación de capas
C. SQLAlchemy no permite leer filas D. Los dataclasses son más rápidos que los objetos ORM

Respuesta: B

13. StaticPool combinado con sqlite:// en las pruebas de la semana 11 se usa para que:

- A. La prueba use PostgreSQL en vez de SQLite B. Las pruebas corran en paralelo
C. Todas las sesiones de la prueba compartan la misma conexión en memoria y vean las mismas tablas D. Se active el modo de solo lectura

Respuesta: C

14. Con cascade="all, delete-orphan" en las relaciones comments e history, ¿qué ocurre al eliminar un Ticket mediante Session.delete() y procesar la unidad de trabajo?

- A. Al borrar el Ticket desde la sesión de SQLAlchemy, sus comentarios e historial se eliminan también B. Los comentarios se crean automáticamente
C. Los comentarios se ordenan alfabéticamente D. Se activa el ON DELETE CASCADE de PostgreSQL automáticamente aunque no esté en el schema

Respuesta: A

15. ¿Qué función cumple tickets.requester_id como FOREIGN KEY hacia users.id?

- A. Garantiza que cada solicitante tenga un solo ticket. B. Impide repetir estados.
C. Sustituye la clave primaria de tickets. D. Exige que el usuario referenciado exista cuando el valor no es NULL.

Respuesta: B



UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA

FACULTAD DE INGENIERÍA EN SISTEMAS DE INFORMACIÓN

SERIE I Continuación

B. Verdadero o falso. Preguntas 16 a 19, 0.20 puntos cada una (0.80 puntos).

16. V / F: Que un repositorio en memoria conserve referencias compartidas garantiza que cualquier implementación SQL persista los cambios sin una operación explícita de actualización.

Respuesta: F Justificación si es falsa: Los repositorios SQL requieren una

Actualización explícita (update () o commit ()) para guardar cambios en la base de datos, a diferencia de la memoria RAM.

17. V / F: Si TicketStatus define los valores "open" y "closed", TicketStatus("Open") reconoce automáticamente "open", aunque no se implemente conversión de mayúsculas.

Respuesta: F Justificación si es falsa: Los Enum de Python evalúan coincidencias exactas

Y sensibles a mayúsculas/minúsculas, por lo que "open" lanza un ValueError.

18. V / F: RecordingNotifier permite comprobar en pruebas las notificaciones registradas sin depender de la salida de consola.

Respuesta: V Justificación si es falsa: Almacena los mensajes de una lista interna en

Memoria, permitiendo que las pruebas afirmen y verifiquen las notificaciones enviadas directamente.

19. V / F: El mapeo de puertos 5433:5432 impide conectarse a PostgreSQL desde el host, incluso usando el puerto 5433.

Respuesta: F Justificación si es falsa: El mapeo 5433:5432 justamente abre y redirecciona

El puerto 5433 del host hacia el puerto 5432 del contenedor, permitiendo la conexión.

C. Respuesta corta. Preguntas 20 a 22, 0.20 puntos cada una (0.60 puntos).

20. Un repositorio devuelve un Ticket de dominio separado de la entidad ORM. ¿Por qué el servicio debe comunicar sus cambios mediante update(ticket)?

Porque la entidad de dominio está desacoplada del ORM; update (ticket) es necesario para remapear los cambios
 A la base de datos.

21. ¿Por qué leer un objeto modificado dentro de la misma sesión no demuestra persistencia después de finalizar la transacción? Indique cómo lo comprobaría.

Porque la sesión mantiene el objeto en memoria RAM; se comprueba cerrando la sesión (session close()) y
 Y consultándolo en una sesión nueva.

22. ¿Qué diferencia hay entre crear el esquema con database.sql en PostgreSQL y Base.metadata.create_all(engine) sobre SQLite? ¿Esta última llamada migra automáticamente tablas existentes?

D. Complete. Preguntas 23 a 25, 0.20 puntos cada una (0.60 puntos).

23. Una clase que hereda de ABC y conserva métodos decorados con @abstractmethod sin implementar no puede instanciarse.

24. El método de Session que envía cambios pendientes a la base de datos sin confirmar la transacción se llama flush() (o flush).

25. La cascada de borrado de una FOREIGN KEY se declara con ON DELETE CASCADE ; la cascada de una relationship se configura mediante el parámetro cascade .



UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA

FACULTAD DE INGENIERÍA EN SISTEMAS DE INFORMACIÓN

SERIE II Práctica sobre HelpDesk EDU 10 puntos

Cada ejercicio vale 2 puntos. Instrucciones adicionales: Para estas pruebas debes crear un nuevo repositorio en tu cuenta de github, ahí deberas generar al menos la siguiente estructura. Esta estructura debes quedar para todos los ejercicios ya resueltos y compartir en link del repositorio al final de este documento y toda la evidencia respectiva.

```
Main
|
|--merge a main. <---
|--Feature_etiquetasEcapsuladas
|---commits respectivos
|
```

1. Etiquetas y encapsulamiento 2 puntos

Archivos: app/models/entities.py y app/domain/errors.py

Agregue a Ticket una lista interna _tags con field(default_factory=list, init=False, repr=False). Exponga tags mediante una propiedad de solo lectura que devuelva una tupla. Implemente add_tag(tag): aplique strip().lower(), rechace valores vacíos con ValidationError y evite duplicados. La asignación ticket.tags = [...] debe fallar. Se evalúa encapsulamiento mediante la interfaz pública; las etiquetas se mantienen en memoria y no requieren migración SQL.

Pruebas y evidencia: Pruebe normalización y duplicados, rechazo de espacios en blanco, independencia entre dos tickets y rechazo de reasignación pública.

Calificación: Colección por instancia y propiedad protegida: 0.60; validación, normalización y duplicados: 0.90; pruebas: 0.50.

2. Observadores y relaciones entre objetos 2 puntos

Archivos: app/services/tickets.py

Implemente TicketService.watchers(ticket_id) -> list[User]. Devuelva el solicitante y, si existe, el técnico asignado; no repita usuarios con el mismo id. Use self.require(ticket_id) y self._users.require(id), sin acceder directamente a repositorios de otros servicios.

Pruebas y evidencia: Compruebe un observador sin técnico, dos con técnico distinto y propagación del error de ticket inexistente. Verifique deduplicación con datos controlados si las reglas permiten el mismo usuario.

Calificación: Recuperación mediante servicios: 0.50; lista correcta sin duplicados: 1.00; pruebas: 0.50.

3. Excepciones y polimorfismo 2 puntos

Archivos: app/domain/errors.py, app/services/tickets.py y app/services/notifications.py

Cree DuplicateAssignmentError como subclase de DomainError. En assign(), rechace una asignación al técnico que el ticket ya tiene, antes de modificar historial o emitir notificaciones. Mantenga las validaciones actuales. Cree WebhookNotifier como implementación del contrato Notifier; simule el canal almacenando los argumentos recibidos en self.sent_payloads, sin HTTP ni impresión. Inyéctelo por el parámetro notifier existente; no agregue condicionales por tipo en TicketService.

Pruebas y evidencia: Compruebe la excepción al reasignar y que no cambien historial ni notificaciones; ejecute una operación válida que notifique y verifique su payload.

Calificación: Excepción y ausencia de efectos adicionales: 0.80; notificador e inyección polimórfica: 0.70; pruebas: 0.50.



UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA

FACULTAD DE INGENIERÍA EN SISTEMAS DE INFORMACIÓN

SERIE II Continuación

4. SQL e integridad referencial 2 puntos

Archivos: docs/database/queries_parcial2.sql

Escriba cuatro consultas sobre el esquema y los datos del proyecto: a) tickets abiertos con el nombre del solicitante mediante JOIN; b) conteo de tickets por técnico asignado, agrupado por id y nombre, con HAVING para excluir conteos cero y orden descendente; c) tickets sin comentarios mediante NOT EXISTS o LEFT JOIN; d) demostración de ON DELETE CASCADE sobre el historial, dentro de BEGIN y ROLLBACK. Use un ticket con historial y muestre un conteo inicial mayor que cero, cero tras DELETE y recuperación del conteo original tras ROLLBACK. Use la tabla de historial real de su esquema.

Pruebas y evidencia: Ejecute las cuatro consultas en PostgreSQL, guarde sus salidas y demuestre que los datos permanecen intactos al finalizar.

Calificación: Consultas a, b y c: 0.40 cada una; demostración completa de cascada y reversión: 0.60; evidencias: 0.20.

5. Consulta agregada y persistencia con SQLAlchemy 2 puntos

Archivos: app/repositories/sqlalchemy.py y tests/

Agregue count_by_status() -> dict[str, int] a SQLAlchemyTicketRepository mediante select(TicketORM.status, func.count()).group_by(TicketORM.status). No cambie la interfaz abstracta TicketRepository. Devuelva solo estados presentes y {} para una base sin tickets. Cree tres tickets en dos estados válidos del proyecto: dos en un estado y uno en otro. Confirme la transacción, cierre esa sesión y consulte el reporte desde una sesión nueva sobre el mismo engine SQLite en memoria con StaticPool.

Pruebas y evidencia: Compruebe el diccionario completo de conteos, la suma de tres tickets y el resultado vacío usando una base de prueba independiente. No use una transición prohibida para preparar los datos.

Calificación: Agregación SQL y contrato: 1.00; prueba desde sesión nueva tras commit: 0.70; caso vacío y evidencia de pruebas: 0.30.

Entrega: código actualizado, pruebas, queries_parcial2.sql y README_parcial2.md con instrucciones reproducibles, resultados y limitaciones. Los incumplimientos parciales se califican con la rúbrica de cada ejercicio.